SFU SIMON FRASER UNIVERSITY

Tutorial 3: Code Development in Flexodeal

These slides are available at www.github.com/javieralmonacid/flexodeal-tutorials

Javier Almonacid

Research Assistant @ NML

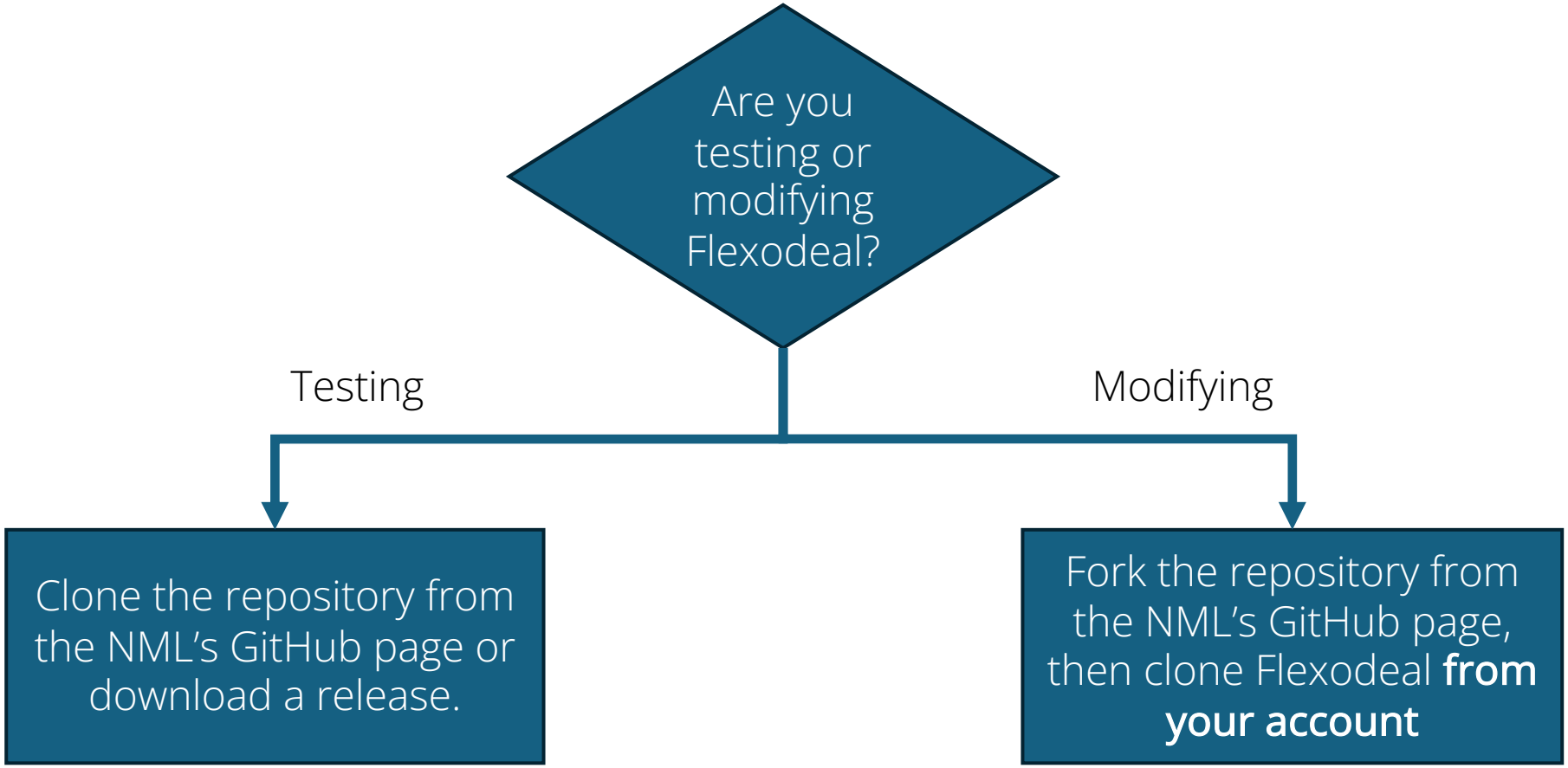
March 20, 2026

Brief Recap of Previous Tutorials

- In Tutorial 1, we discussed the main details of the Flexodeal framework. We discussed how different types of contractions can be studied using Flexodeal Lite.
- In Tutorial 2, we turned our focus to Flexodeal and its default experiment. We also discussed how working with non-standard meshes requires the use of a quadrature point file.

In essence, we have discussed how to *use* Flexodeal. We will now focus on how to **modify** Flexodeal.

Recommended Workflow



Forking a Repository

If you clone Flexodeal from the NML's GitHub page:

- You can only perform local changes.
- You cannot create remote branches (local branches can still be created).
- You cannot track your changes.

If you fork Flexodeal, and then clone this repository from your account:

- Your can create multiple branches for different projects and track your changes.
- You can fix the version of Flexodeal you'd like to use.

Why Should You Track Your Changes?

Reproducibility	• Recover the exact code used for results. • Re-run experiments months (or years) later. • Ensure consistency across papers and collaborators.
Debugging & Validation	• Identify *when* and *why* results changed. • Compare working vs broken versions. • Safely experiment without losing stable code.
Collaboration	• Share clear, traceable contributions. • Enable others to review and build on your work. • Avoid overwriting or losing teammates' changes.
Project Management	• Keep features and experiments organized. • Work on multiple ideas in parallel (via branches). • Maintain a clean, structured codebase.
Long-term Research Value	• Preserve your work beyond a single project. • Make your research transparent and credible. • Support future extensions of your work.

Main Goal

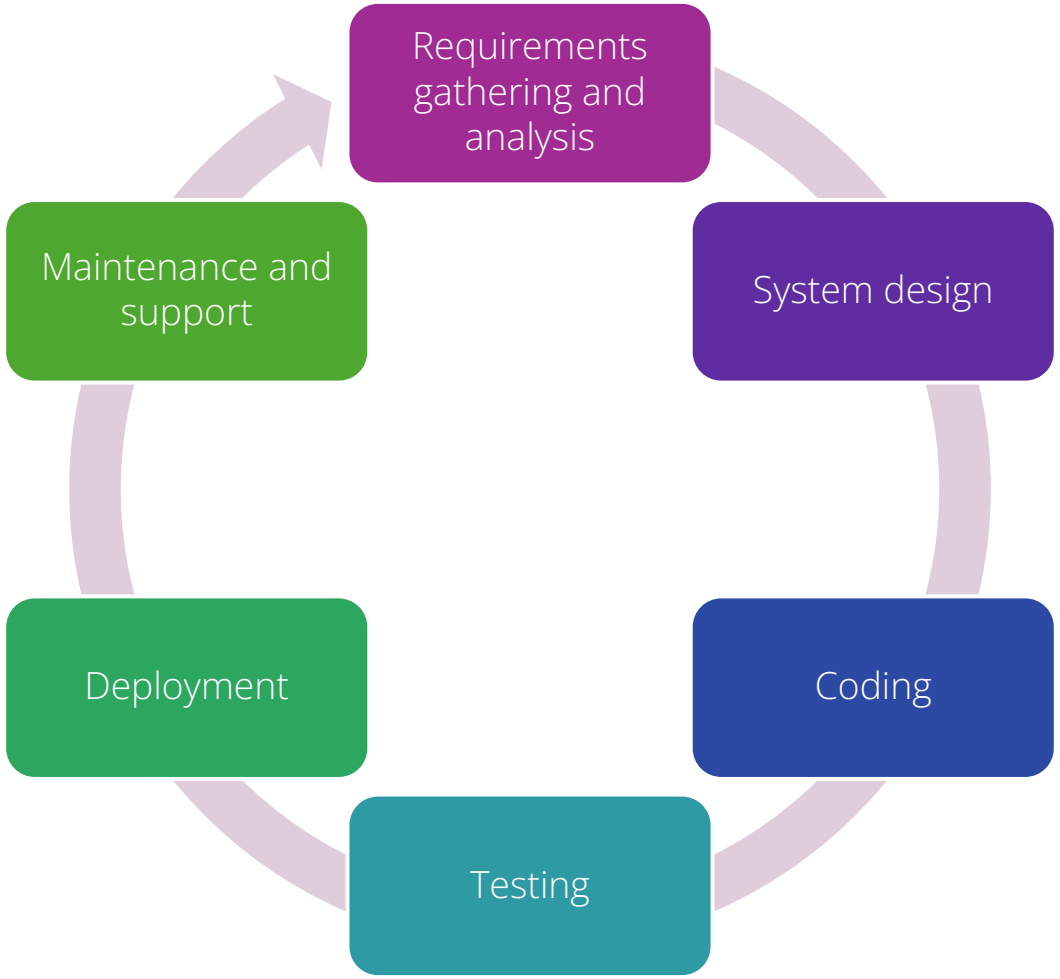
- We wish to include intramuscular fat in Flexodeal Lite. Since this is already included in Flexodeal, we take relevant design from there and port it over to Flexodeal Lite.
- We will do this in a tracked setting, starting from the latest version of Flexodeal Lite.

Introducing Intramuscular Fat in Flexodeal Lite

Software Development Life Cycle (SDLC)

- As with any piece of software, any new feature in Flexodeal should be designed and implemented with care.

- The SDLC is a ubiquitous framework for managing an entire software project from start to finish, ensuring consistency, efficiency, and quality.



https://github.com/resources/articles/what-is-sdlc

Requirements Gathering and Analysis

Objective	• Capture the effect of intramuscular fat on active muscle dynamics.
Modelling Approach	• Homogenized strain-energy function • Fat is assumed to be isotropic, quasi-incompressible, and uniformly distributed.
Inputs	• Fat fraction • Parameters that define the strain-energy function for fat
Validation Targets	• Decreasing force generation with increasing fat fraction. • Decrease proportional to the fat fraction in the context of an isometric contraction of parallel fibred muscle.
Constraints	• Backwards compatible (zero fat fraction should recover original behaviour). • Minimal computational overhead.
Out of Scope	• Spatial heterogeneity (clumps). • Viscoelastic model

System Design: Mathematical Details

- Overall decomposition of the strain-energy function (SEF)

$$\Psi = \Psi_{vol} + \Psi_{iso}, \tag{1}$$

- Volumetric strain energy:

$$\Psi_{vol} = \frac{(1-\chi_{fat})\kappa_{mus} + \chi_{fat}\kappa_{fat}}{4}(J^2 - 2\ln J - 1), \tag{2}$$

- Isochoric strain energy:

$$\Psi_{iso} = (1-\chi_{fat})\Big(\Psi_{mus}(\bar{I}_4;\dot{\bar{I}}_4) + \Psi_{bm}(\bar{I}_1)\Big) + \chi_{fat}\Psi_{fat}(\bar{I}_1), \tag{3}$$

- Fictitious Kirchhoff stress:

$$\bar{\boldsymbol{\tau}} = (1-\chi_{fat})\Big(\bar{\boldsymbol{\tau}}_{fibre} + \bar{\boldsymbol{\tau}}_{bm}\Big) + \chi_{fat}\bar{\boldsymbol{\tau}}_{fat}, \tag{4}$$

$$\bar{\boldsymbol{\tau}}_{fat} = 2s_{fat}\Big(3c_3(\bar{I}_1-3)^2 + 2c_2(\bar{I}_1-3) + c_1\Big)\bar{\mathbf{b}}, \tag{5}$$

- Elasticity tensors:

$$\bar{\mathbb{C}} = (1-\chi_{fat})\Big(\bar{\mathbb{C}}_{fibre} + \bar{\mathbb{C}}_{bm}\Big) + \chi_{fat}\bar{\mathbb{C}}_{fat}, \tag{6}$$

$$\bar{\mathbb{C}}_{fat} = 4s_{fat}\Big(6c_3(\bar{I}_1-3) + 2c_2\Big)\bar{\mathbf{b}}\otimes\bar{\mathbf{b}}. \tag{7}$$

Therefore, 6 new parameters need to be included (with the following default values):

- Bulk modulus of fat $\kappa_{fat} = 10^7$ Pa,
- Fat fraction $\chi_{fat} = 0.02$,
- Yeoh constants $c_1 = 323.91$, $c_2 = 5163.1$, and $c_3 = -3872.9$,
- A "fudge factor" $s_{fat} = 1$.

Source: R. N. Konno, N. Nigam, J. M. Wakeling, and S. A. Ross, *The contributions of extracellular matrix and sarcomere properties to passive muscle stiffness in cerebral palsy*, Frontiers in Physiology, 12 (2022), p. 2498.

Quantities in blue are new to the code

Coding

First, clone the repository and set your tracking environment

Description	UNIX Command
Fork Flexodeal Lite from the NML's GitHub page	-
Navigate to a convenient directory, then clone **your forked version** of Flexodeal Lite	`git clone https://github.com/<your-username>/flexodeal-lite.git`
Navigate to the Flexodeal Lite folder.	`cd flexodeal-lite`
Create a new branch, let's call it fat_implementation	`git checkout –b fat_implementation`
Link your local branch to GitHub	`git push –u origin fat_implementation`

Important: Because you are cloning the repository using the https address, "push" operations will require you to enter your username and "password" for GitHub. Note that your password here is given by one of your **Personal Access Tokens (PAT)**, which you can obtain from your account. To do this, go to your profile, then Settings -> Developer Settings -> Personal Access Tokens -> Tokens (classic) -> Generate New Token. Make sure to give "repo" permissions. To avoid being asked for username and password every time you push changes, sync your machine's SSH key with your GitHub account.

Coding

- Compile Flexodeal Lite, then run an isometric contraction (the one in the **`examples`** folder) and save these results (you will need them for the testing phase).

```
./flexodeal-lite -PARAMETERS=examples/IC_parameters.prm
-BDY_STRAIN=examples/IC_strain.dat -ACTIVATION=examples/IC_activation.dat
-OUTPUT_DIR=results-original
```

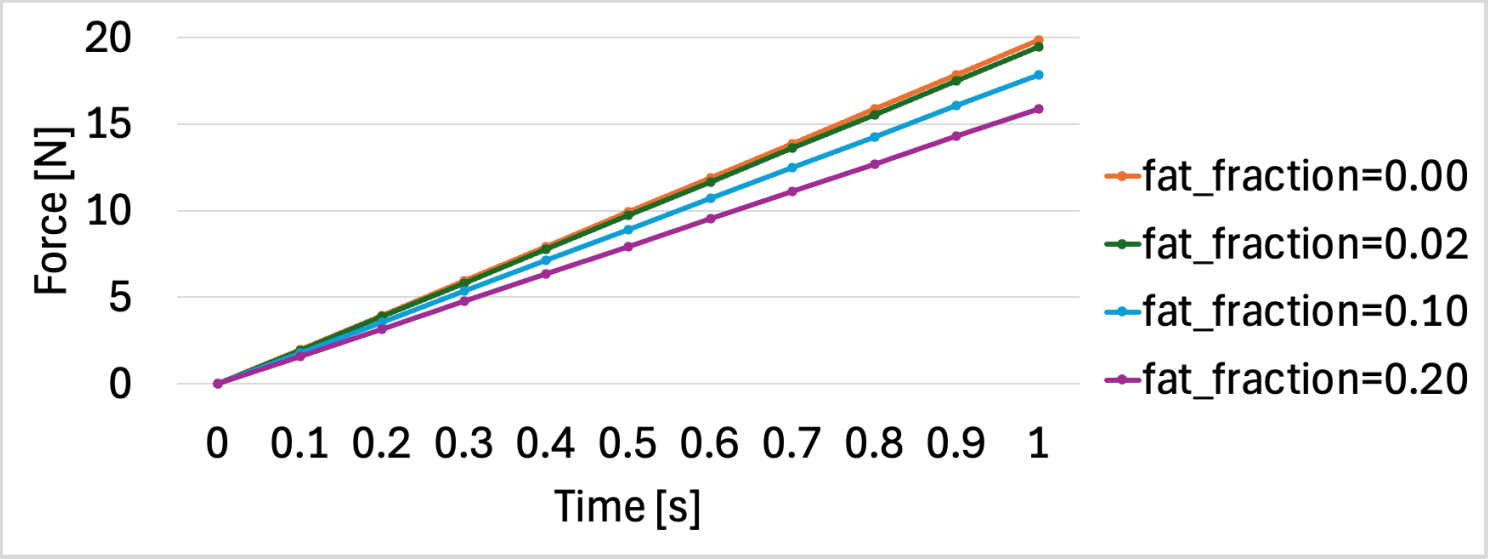
- Open the folder **`flexodeal-lite`** in Visual Studio Code.
- Modify the files **`parameters.prm`** and **`flexodeal-lite.cc`** according to the instructions in the file **`T3_SYSTEM_DESIGN.md`**, located in the `flexodeal-tutorials` repository.
- You may replace the contents of the file **`flexodeal-lite.cc`** by those of the file **`flexodeal-lite-with-fat.cc`**, also available in the `flexodeal-tutorials` repository.

Testing

- Run 4 simulations with the following fat fractions: 0.00, 0.02, 0.10, 0.20.
- Using the file `force_data-3d.csv` for each simulation:
  - Verify that the output for fat fraction = 0 matches the output of the unmodified code.
  - Verify that, as the fat fraction increases, the total force at either one of the fixed ends decreases.

Force (N) at the +X Fixed End

Time [s]	Original	fat_fraction=0.00	fat_fraction=0.02	fat_fraction=0.10	fat_fraction=0.20	rel_change 0.00	rel_change 0.02	rel_change 0.10	rel_change 0.20
0	6.3239E-16	6.3239E-16	8.7200E-16	8.6031E-16	6.0300E-16				
0.1	1.9825E+00	1.9825E+00	1.9430E+00	1.7847E+00	1.5900E+00	0.00%	-1.99%	-9.98%	-19.80%
0.2	3.9650E+00	3.9650E+00	3.8859E+00	3.5694E+00	3.1700E+00	0.00%	-1.99%	-9.98%	-20.05%
0.3	5.9475E+00	5.9475E+00	5.8288E+00	5.3541E+00	4.7600E+00	0.00%	-2.00%	-9.98%	-19.97%
0.4	7.9301E+00	7.9301E+00	7.7720E+00	7.1391E+00	6.3500E+00	0.00%	-1.99%	-9.97%	-19.93%
0.5	9.9132E+00	9.9132E+00	9.7154E+00	8.9243E+00	7.9300E+00	0.00%	-2.00%	-9.98%	-20.01%
0.6	1.1897E+01	1.1897E+01	1.1659E+01	1.0710E+01	9.5200E+00	0.00%	-2.00%	-9.98%	-19.98%
0.7	1.3881E+01	1.3881E+01	1.3604E+01	1.2496E+01	1.1100E+01	0.00%	-2.00%	-9.98%	-20.03%
0.8	1.5866E+01	1.5866E+01	1.5550E+01	1.4283E+01	1.2700E+01	0.00%	-1.99%	-9.98%	-19.95%
0.9	1.7853E+01	1.7853E+01	1.7496E+01	1.6071E+01	1.4300E+01	0.00%	-2.00%	-9.98%	-19.90%
1	1.9841E+01	1.9841E+01	1.9445E+01	1.7861E+01	1.5900E+01	0.00%	-2.00%	-9.98%	-19.86%



Do these results make sense?

Git Workflow: Add, Commit, Push

Step 1: `git add` (staging)

- Select which files to include.
- Can stage:
  - Specific files, for example, `git add flexodeal-lite.cc parameters.prm`
  - All files: `git add .`
- Think: "What do I want in this update?"

Step 2: `git commit` (local save)

- Creates a snapshot of staged changes.
- Write clear, concise messages: `git commit -m "Added intramuscular fat"`
- Think: "What did I change and why?"

Step 3: `git push` (Share)

- Upload commits to GitHub.
- Makes changes visible to collaborators.

⚠️ Good practices

- Commit small, logical changes.
- Use meaningful commit messages.
- Push regularly (don't accumulate too much).

⚠️ If this is your **first time commiting your changes**, you might be asked to provide your **name** and **email address**.

Deployment

- Deployment is the process of making new code usable by others.
- Goes beyond coding. It includes integration, documentation, and reproducibility. In our example, this could mean:
  - Updating all the .prm files.
  - Updating any README.md files.
  - Creating further examples that may be used as tests.
  - Releasing a new version.
- This is particularly important if:
  - You're planning to submit a pull request (PR) to include your changes in the main Flexodeal repository.
  - You're planning to share your code publicly.

Contributing to Flexodeal: Key Questions

Before opening a pull request (PR), or even before thinking of adding a new feature to Flexodeal, ask:

- Is this feature **broadly useful**? That is, not just for a single project or boundary conditions.
- Is it **generalizable**? That is, is it independent of specific geometry or boundary conditions.
- Is it **testable**? Can you verify **correctness** with a clear test case?
- Is it **reproducible**? Can others run and obtain consistent results?
- Is it **documented**?

Maintenance & Support

- This is ongoing work after deployment to ensure any new feature remains correct, stable, and useful over time.
- Typical work includes bug fixes or developing new test cases based on new data or literature.
- **Always prevent regressions**.

Best Software Development Practices

Write clear, readable code

- Use meaningful names for your variables and keep a simple structure.

Keep changes small and focused.

- One feature or fix per commit.

Document your work

- Explain what your code does and why.

Test your code

- Verify (and, if possible, benchmark) your results and avoid breaking existing functionality.

Use version control (Git)

- Track changes and work safely with others.

Reuse, don't duplicate

- Build on existing code instead of rewriting (unless you have a very good reason to do so).

More Resources

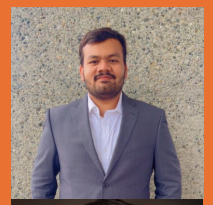
- Finite element simulation using deal.II: https://www.dealii.org/current/doxygen/deal.II/Tutorial.html
- Mathematical details of Flexodeal:
  - Almonacid, J. A., Domínguez-Rivera, S. A., Konno, R. N., Nigam, N., Ross, S. A., Tam, C., & Wakeling, J. M. (2024). A three-dimensional model of skeletal muscle tissues. *SIAM journal on applied mathematics*, *84*(3), S538-S566.
  - Almonacid Paredes, J. A. (2025). On the nonlinear elastodynamics of skeletal muscle (Chapters 5 and 6). PhD Thesis, Simon Fraser University (available at www.sfu.ca/~Javiera).
- ParaView documentation: https://docs.paraview.org/en/v5.13.1/UsersGuide/introduction.html

www.github.com/sfu-nml

www.linkedin.com/company/sfu-nml

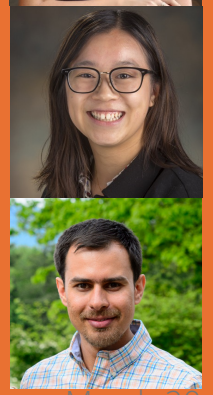
Flexodeal contributors

Javier Almonacid

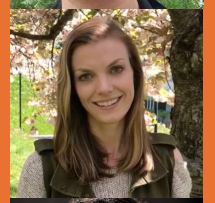

Harshil Pathak

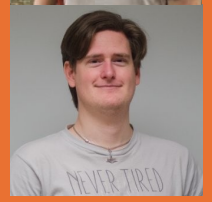
Kshitij Patil

Ryan N. Konno

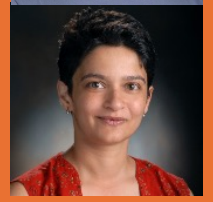

Cassidy Tam

Sebastian Dominguez


Stephanie A. Ross


David S. Ryan

Hadi Rahemi


Nilima Nigam

James M. Wakeling